DATABASE MANAGEMENT SYSTEM

MINI PROJECT REPORT

ROOMROUTE

Nagpur Rental Search and Listing Platform

DBMS PROJECT · POSTGRESQL / SQLITE PREVIEW

Prepared by: Tanay Gaykwad

College / department: _____

Roll number: _____

Repository link: https://github.com/tanay-gaykwad/roomroute

Live application: Not deployed

This report documents the current code and local synthetic preview. It does not claim that fictional records are real rentals or that a public service is live.

Prepared 22 October 2026 · Nagpur, Maharashtra

INDEX

Sr.	Contents	Page
1	Project Structure / 20-Step Sequence	3
2	Aim, Objectives, Theory & Requirements	4
3	Original PostgreSQL Code	5–11
4	Basic SQL Query Outputs	12
5	JOIN Outputs	13
6	Aggregate Functions, GROUP BY & Subquery	14
7	Indexes and database behavior	15
8	CRUD — Create & Read	16
9	CRUD — Update & Delete	17
10	Frontend interface illustrations	18–20
11	Result & Conclusion	21
12	Advantages, Disadvantages & Future Scope	21

Report layout follows the supplied college-format DBMS report. Database examples are adapted to the RoomRoute schema and features actually present in this repository.

PROJECT STRUCTURE / 20-STEP SEQUENCE

Step	Module
1	Create owner and renter profile tables
2	Create account and role table
3	Create verified-session table
4	Create one-time authentication-token table
5	Create rental listing table
6	Create inquiry table
7	Create moderation report table
8	Create durable email outbox table
9	Create authentication-attempt records
10	Create indexes and schema migration version
11	Load synthetic preview owners and renters
12	Load synthetic preview room and flat listings
13	Run basic filter and sort queries
14	Join listings to owners
15	Aggregate listing and inquiry counts
16	Group by area and property type
17	Use bounded listing pagination
18	Create an owner listing (CRUD)
19	Approve, pause, report and export records
20	Review frontend and local service flows

1. AIM

To design and implement RoomRoute, a Nagpur-focused rental search platform that models owners, renters, accounts, room and flat listings, inquiries, and moderation using relational database operations.

2. OBJECTIVES

• Store owner, renter, account, listing, inquiry and moderation data with relational keys. • Search rooms and flats using location, budget, utilities, furnishing and parking. • Apply authentication, verification, inquiry, listing review and privacy workflows. • Demonstrate filtering, joins, aggregation, indexes and CRUD with synthetic preview data.

3. THEORY

A relational database organizes records into tables and links them with primary and foreign keys. Constraints preserve valid relationships; indexes support common lookups; SQL performs filtering, joins, grouping and updates. RoomRoute uses PostgreSQL in configured service mode and SQLite for local preview. Synthetic records are explicitly separated from registered accounts and real user submissions.

4. SOFTWARE & HARDWARE REQUIREMENTS

Category	Requirement
Software	Python 3.12+, PostgreSQL, Redis, modern browser; Docker Compose for local multi-service preview
Libraries	argon2-cffi, psycopg with pool support, redis-py
Hardware	Development computer with internet for package setup; deployment resources must be selected and load-tested
Current evidence	Local preview database contains 50 synthetic owners, 200 synthetic renter profiles and 50 synthetic listings; no account or inquiry rows

5. ORIGINAL POSTGRESQL CODE — ACCOUNTS AND AUTHENTICATION

RoomRoute's schema is created through additive migrations in server.py. The snippets below summarize the current PostgreSQL table definitions.

```
CREATE TABLE accounts (
  id TEXT PRIMARY KEY,
  email TEXT NOT NULL UNIQUE,
  name TEXT NOT NULL,
  role TEXT NOT NULL CHECK
    (role IN ('owner','student','employee','moderator')),
  password_hash TEXT NOT NULL,
  is_verified INTEGER NOT NULL DEFAULT 0,
  is_active INTEGER NOT NULL DEFAULT 1,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE sessions (
  token_hash TEXT PRIMARY KEY,
  account_id TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  csrf_token TEXT NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Passwords are stored as Argon2 hashes. Session tokens are random; only their digest is stored. Verification and reset tokens are hashed, expiring and single use.

5. ORIGINAL POSTGRESQL CODE — OWNER, RENTER AND LISTING TABLES

```
CREATE TABLE owners (
  id TEXT PRIMARY KEY, name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  role TEXT NOT NULL DEFAULT 'owner',
  is_sample INTEGER NOT NULL DEFAULT 0
);
CREATE TABLE users (
  id TEXT PRIMARY KEY, name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE, role TEXT NOT NULL,
  occupation TEXT NOT NULL, budget INTEGER NOT NULL,
  is_sample INTEGER NOT NULL DEFAULT 0
);
CREATE TABLE listings (
  id TEXT PRIMARY KEY,
  owner_id TEXT NOT NULL REFERENCES owners(id),
  title TEXT NOT NULL, type TEXT NOT NULL,
  rent INTEGER NOT NULL, deposit INTEGER NOT NULL,
  area TEXT NOT NULL, city TEXT NOT NULL DEFAULT 'Nagpur',
  furnishing TEXT NOT NULL, parking INTEGER NOT NULL,
  electricity_included INTEGER NOT NULL,
  available_from TEXT NOT NULL, photo TEXT NOT NULL,
  summary TEXT NOT NULL, amenities TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'pending',
  is_sample INTEGER NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Foreign keys link listings to owners. New real listings start in pending review; only active records appear in public search. Real listings have no photo upload yet.

5. ORIGINAL POSTGRESQL CODE — INQUIRIES AND MODERATION

```
CREATE TABLE inquiries (
  id BIGSERIAL PRIMARY KEY,
  listing_id TEXT NOT NULL REFERENCES listings(id),
  student_id TEXT NOT NULL REFERENCES users(id),
  message TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'new',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE listing_reports (
  id BIGSERIAL PRIMARY KEY,
  listing_id TEXT NOT NULL REFERENCES listings(id),
  reporter_id TEXT NOT NULL REFERENCES accounts(id),
  reason TEXT NOT NULL, details TEXT NOT NULL DEFAULT '',
  status TEXT NOT NULL DEFAULT 'open',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE email_outbox (
  id BIGSERIAL PRIMARY KEY,
  recipient TEXT NOT NULL, subject TEXT NOT NULL,
  body TEXT NOT NULL, attempts INTEGER NOT NULL DEFAULT 0,
  next_attempt TIMESTAMPTZ NOT NULL DEFAULT now(),
  sent_at TIMESTAMPTZ, last_error TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Inquiry delivery is queued in the database and requires a configured SMTP provider in service mode. The local preview can print verification links to its terminal when email is not configured.

5. ORIGINAL POSTGRESQL CODE — SYNTHETIC PREVIEW DATA

```
# Local-only operator command; idempotent and removable.
python server.py --seed-sample-data

# Remove only tagged synthetic records.
python server.py --remove-sample-data

# Production startup refuses these sample-data operations.
```

The repository's local SQLite fixture currently has 50 owners, 200 renter profiles and 50 listings. These records have no account credentials and sample listings cannot receive inquiries.

This project does not include an SQL seed dump. The fixture is generated by the application's explicit seed command, and production startup blocks seeding. This prevents a fictitious marketing claim that the example profiles are real owners or renters.

5. ORIGINAL POSTGRESQL CODE — BASIC SQL AND JOIN

Examples use bound parameters in application code; values below illustrate direct database analysis.

```
-- Public listings within a budget
SELECT id, title, type, area, rent, furnishing
FROM listings
WHERE status = 'active' AND is_sample = 1
  AND rent <= 12000
ORDER BY rent ASC, created_at DESC
LIMIT 50 OFFSET 0;

-- Listing with owner display name
SELECT l.title, l.area, l.rent, o.name AS owner_name
FROM listings AS l
JOIN owners AS o ON o.id = l.owner_id
WHERE l.status = 'active'
ORDER BY l.created_at DESC;

-- Inquiries linked to their listing
SELECT l.title, i.status, i.created_at
FROM inquiries AS i
JOIN listings AS l ON l.id = i.listing_id;
```

Public listing search applies filters in the database and caps each response at 100 records. Owners are scoped to the selected page; the browser can request more results with pagination.

5. ORIGINAL POSTGRESQL CODE — AGGREGATES AND GROUPING

```
SELECT COUNT(*) AS listing_count
FROM listings WHERE is_sample = 1;

SELECT area, COUNT(*) AS places,
       MIN(rent) AS lowest_rent,
       ROUND(AVG(rent)) AS average_rent
FROM listings
WHERE is_sample = 1
GROUP BY area
HAVING COUNT(*) >= 2
ORDER BY area;

SELECT type, COUNT(*) AS places
FROM listings WHERE is_sample = 1
GROUP BY type ORDER BY places DESC;

-- A bounded sample of premium listings
SELECT title, rent FROM listings
WHERE is_sample = 1
  AND rent > (SELECT AVG(rent) FROM listings WHERE is_sample = 1)
ORDER BY rent DESC LIMIT 5;
```

These queries are suitable for analyzing the synthetic preview inventory. They are not a rating or review system: student reviews of owners and properties are not implemented.

5. ORIGINAL POSTGRESQL CODE — INDEXES AND MIGRATIONS

```
CREATE INDEX IF NOT EXISTS idx_listings_public_order
  ON listings(status, created_at DESC);
CREATE INDEX IF NOT EXISTS idx_listings_public_area
  ON listings(status, area, created_at DESC);
CREATE INDEX IF NOT EXISTS idx_listings_public_rent
  ON listings(status, rent);
CREATE INDEX IF NOT EXISTS idx_inquiries_listing_order
  ON inquiries(listing_id, id DESC);
CREATE INDEX IF NOT EXISTS idx_reports_status
  ON listing_reports(status, created_at);

CREATE TABLE IF NOT EXISTS schema_migrations (
  version INTEGER PRIMARY KEY,
  applied_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

The supplied college format requests a View and Trigger exercise. RoomRoute currently relies on parameterized queries and explicit application flows; it has no database View or Trigger. This report does not claim those objects exist.

5. ORIGINAL POSTGRESQL CODE — CRUD OPERATIONS

```
-- CREATE: owner-submitted real listings start pending
INSERT INTO listings
  (id, owner_id, title, type, rent, deposit, area,
   furnishing, parking, electricity_included,
   available_from, photo, summary, amenities, status,
   is_sample)
VALUES
  (:id, :owner_id, :title, :type, :rent, :deposit, :area,
   :furnishing, :parking, :electricity_included,
   :available_from, '', :summary, '[]', 'pending', 0);

-- READ: active public listings only
SELECT id, title, rent, area FROM listings
WHERE status='active' ORDER BY created_at DESC;

-- UPDATE: owner withdraws an active listing
UPDATE listings SET status='paused'
WHERE id=:listing_id AND owner_id=:owner_id
  AND is_sample=0 AND status='active';

-- DELETE: privacy action removes linked account data
-- Performed in a transaction after password verification.
```

The live service requires PostgreSQL and Redis. A local SQLite database is used only for development and demonstration.

6. OUTPUT — BASIC SQL QUERIES

Local preview table	Records
Owners (synthetic)	50
Renter profiles (synthetic)	200
Listings (synthetic)	50
Accounts	0
Inquiries	0

Representative query: SELECT COUNT(*) FROM listings WHERE is_sample = 1; returns 50 for the checked local fixture. Sample identities and listings are fictional and cannot be contacted through the inquiry feature.

Example budget search returns listings meeting the selected maximum rent after area and property-type filters. Public responses are capped at 100 rows per page.

7. OUTPUT — JOIN QUERIES

Joined field	Relationship
Listing title and owner name	listings.owner_id → owners.id
Inquiry and place title	inquiries.listing_id → listings.id
Moderation report and reporter	listing_reports.reporter_id → accounts.id

Current preview listings join to 50 synthetic owner rows. The current fixture has no inquiries or platform accounts, so the inquiry and report joins return zero rows until actual verified users submit data.

```
SELECT l.title, o.name, l.area, l.rent
FROM listings l
INNER JOIN owners o ON o.id=l.owner_id
WHERE l.is_sample=1
ORDER BY l.area, l.rent
LIMIT 5;
```

8. OUTPUT — AGGREGATES, GROUPING AND SUBQUERY

Metric from local sample fixture	Result
Synthetic listings	50
Synthetic owners	50
Synthetic renter profiles	200
Active account records	0
Submitted inquiries	0

Use GROUP BY area or type to compare inventory counts and rent ranges. A subquery can select listings above the preview's average rent. Results describe only the fictional local seed data and are not evidence of market availability or demand.

9. OUTPUT — INDEXES, VIEW AND TRIGGER

Indexes present in the application schema support public listing order, area and rent filtering, inquiry lookups, session expiry, authentication attempts, pending email and moderation reports.

Database object	Current project status
idx_listings_public_order	Implemented
idx_listings_public_area	Implemented
idx_listings_public_rent	Implemented
idx_inquiries_listing_order	Implemented
Student-facing Review View	Not implemented
Database trigger	Not implemented

The app currently has no verified student review feature. It would be unsafe to publish unverified accusations about owners; a future review feature needs verified tenancy and dispute handling.

10. CRUD OUTPUT — CREATE AND READ

Owner submission is saved as a pending real listing. Public search only returns records approved by a moderator and marked active.

Operation	Illustrative result
CREATE listing	Saved with status = pending
READ owner workspace	Owner sees their pending listing
READ public catalogue	Pending listing is hidden
READ sample listing	Clearly marked; inquiry disabled

This is an application-flow result. The report does not create records in the owner's database.

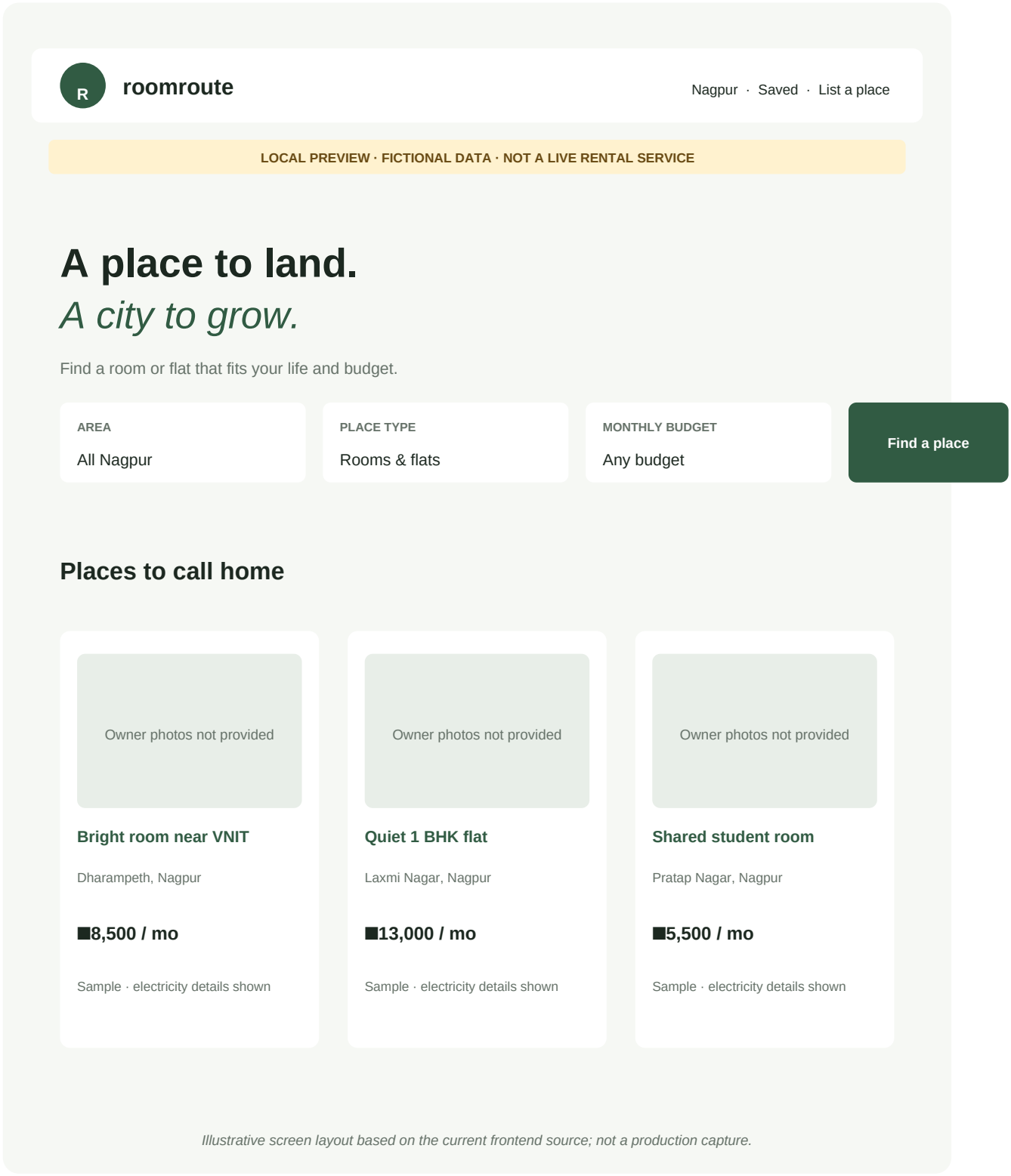
11. CRUD OUTPUT — UPDATE AND DELETE

Operation	Behavior
UPDATE availability	Owner pauses an active real listing
Relist	Listing returns to moderation queue
DELETE account	Password verified; linked records removed
Export account	Authenticated account can download its data

Sample listings cannot be modified through the owner workspace. Account deletion and export endpoints exist, but a complete privacy program still requires operator policies, retention rules and tested backups.

12. FRONTEND APP / WEBPAGE — SEARCH AND DISCOVERY

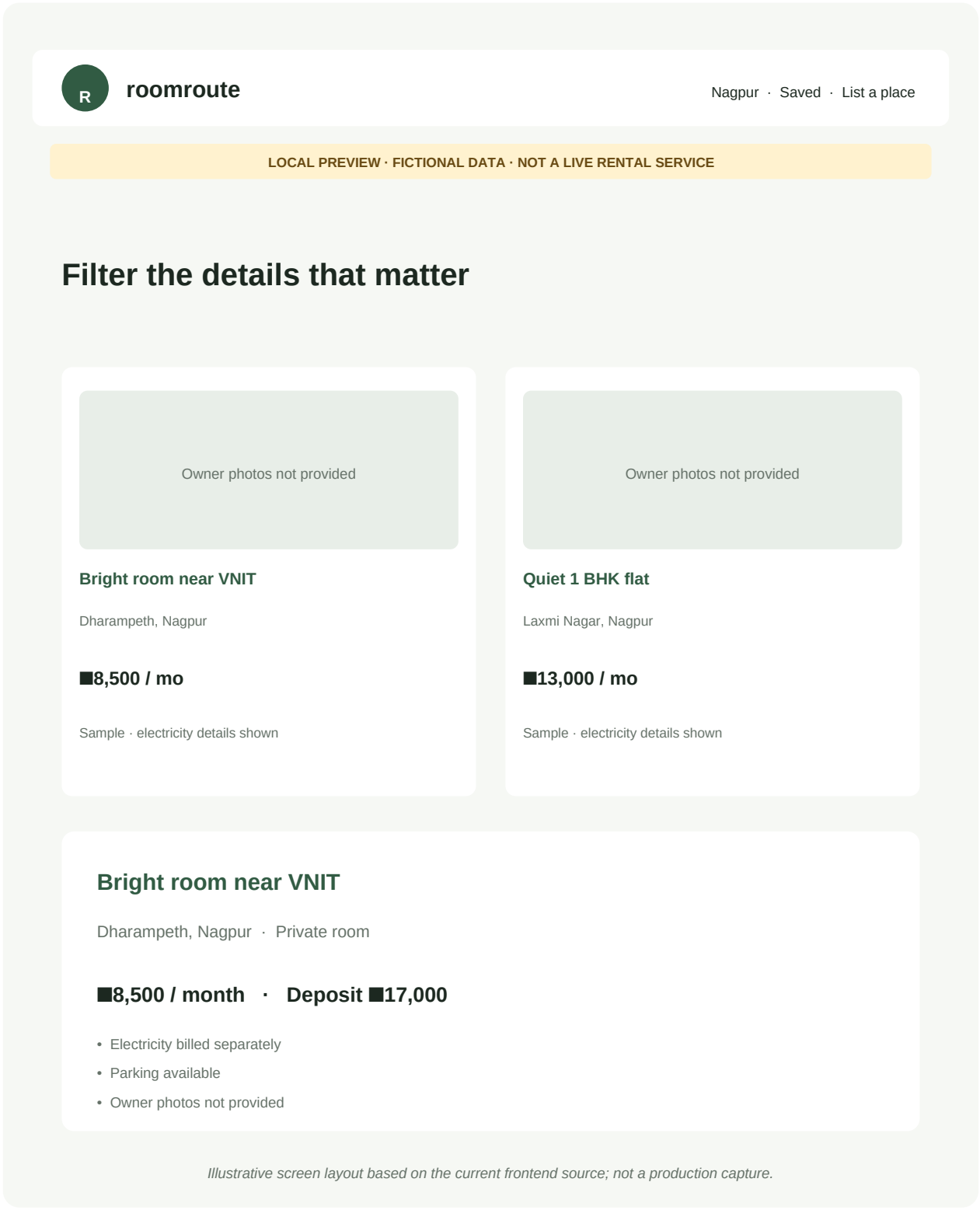
The frontend is responsive HTML, CSS and JavaScript. Search controls expose Nagpur area, property type and monthly budget. Sample inventory is labeled, and owner photos are not fabricated for real listings.



Illustrative screen layout based on the current frontend source; not a production capture.

12. FRONTEND APP / WEBPAGE — LISTING DETAILS

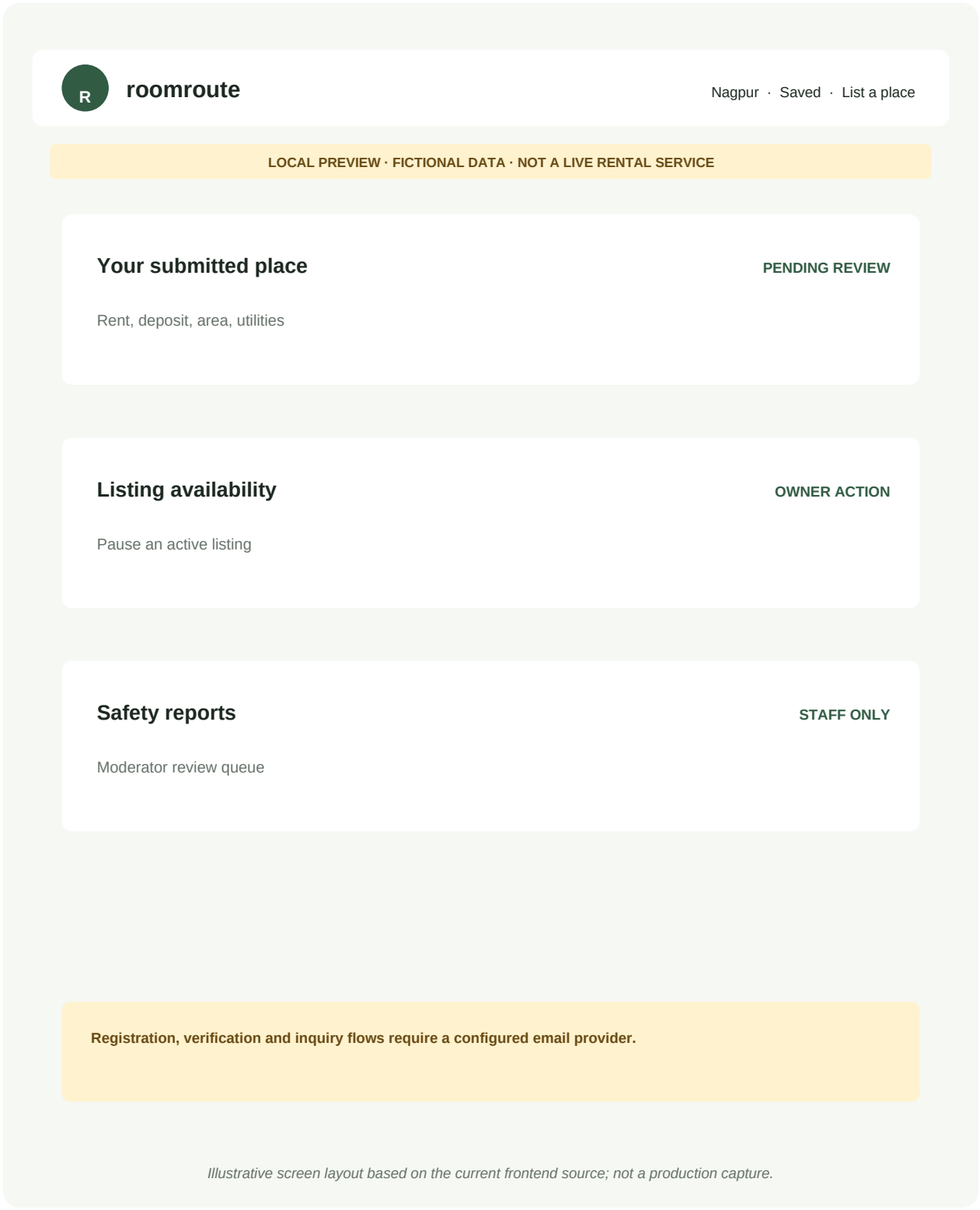
Property cards show rent, deposit context, location, furnishing, parking and electricity details. These report illustrations follow the current frontend source and sample fixture; they are not screenshots captured from a live service.



Illustrative screen layout based on the current frontend source; not a production capture.

12. FRONTEND APP / WEBPAGE — OWNER AND REVIEW FLOWS

Real owner submissions require email verification and moderation before they are public. The current service does not provide property identity checks, photo upload, or editing all listing fields after submission.



Illustrative screen layout based on the current frontend source; not a production capture.

13. RESULT

RoomRoute has a working local-preview web interface and Python backend. The repository implements relational data access, synthetic records, filtering, bounded public listing pages, authenticated account flows, email-verification and password-reset token handling, inquiry storage and notification queue, listing moderation, reports, and account export/deletion routes. The current local fixture contains 50 sample owners, 200 sample renter profiles, and 50 sample listings.

14. CONCLUSION

The project demonstrates how relational databases support a rental discovery workflow. A code-level production mode uses PostgreSQL and shared Redis, but the service is not deployed and has not completed an independent penetration test or live restore test. A free ephemeral deployment is not suitable for real account data.

15. ADVANTAGES

• Combines room and flat search. • Filters by budget and practical property details. • Labels synthetic inventory. • Uses password hashing, email verification, CSRF protections, shared throttling, moderation and privacy endpoints. • Includes Docker Compose and automated CI configuration.

16. DISADVANTAGES / REMAINING WORK

• No public deployment, real listing inventory or provider credentials. • No moderator MFA, owner/property verification, photo upload, listing edit, or student review system. • No measured capacity claim; pagination and load testing remain for private inboxes and reports. • Policies, backups, monitoring, restore drills and independent security review are still needed.

17. FUTURE SCOPE

Add provider-backed image storage, owner verification, moderator MFA, pagination and retention for private records, verified tenancy reviews with dispute controls, AI-assisted search after privacy and quality assessment, and monitored production infrastructure with tested backups.

Repository: https://github.com/tanay-gaykwad/roomroute | Live application: Not deployed

Status reviewed on 22 October 2026. Synthetic data is for interface and database exercises only.